Backtracking

> → **Key Idea**: Explore solution space recursively and backtrack when constraints are violated

> • **When to use**: Permutations, combinations, constraint satisfaction, puzzle solving

> ⊕ **Complexity**: Exponential in worst case, depends on pruning effectiveness

> ⚠ **Common Pitfall**: Not properly backtracking state or missing pruning opportunities

Overview

Backtracking systematically explores all potential solutions by building candidates incrementally and abandoning ("backtracking") as soon as it determines that the candidate cannot lead to a valid solution.

Visual Example

Backtracking Decision Tree (Permutations of [1,2,3])

```
                    []
          /         |         \
        [1]         [2]         [3]
       /   \       /   \       /   \
    [1,2] [1,3]  [2,1] [2,3]  [3,1] [3,2]
      |     |      |     |      |     |
  [1,2,3] [1,3,2] [2,1,3] [2,3,1] [3,1,2] [3,2,1]

Result: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
```

N-Queens Backtracking

```
N=4 Solution Process:

Step 1:    Step 2:    Step 3:    Step 4 (backtrack):
Q . . .    Q . . .    Q . . .    Q . . .
. . . .    . . Q .    . . Q .    . . . Q
. . . .    . . . .    . Q . .    . Q . .
. . . .    . . . .    . . . .    . . Q .  ✓ Valid!

Place Q → Check conflicts → Backtrack if invalid → Try next column
```

Subsets Generation

```
Input: [1, 2, 3]

Decision tree (include/exclude each element):
                    []
            /               \
          []                [1]
        /    \            /     \
      []     [2]        [1]     [1,2]
     / \     / \        / \      / \
   [] [3] [2] [2,3] [1] [1,3] [1,2] [1,2,3]

Result: [[], [3], [2], [2,3], [1], [1,3], [1,2], [1,2,3]]
```

Core Implementation

Permutations Template

```java
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, new ArrayList<>(), new boolean[nums.length], result);
    return result;
}

private void backtrack(int[] nums, List<Integer> path, boolean[] used,
                       List<List<Integer>> result) {
    if (path.size() == nums.length) {
        result.add(new ArrayList<>(path));
        return;
    }

    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;

        path.add(nums[i]);
        used[i] = true;

        backtrack(nums, path, used, result);

        path.remove(path.size() - 1);  // Backtrack
        used[i] = false;
    }
}
```

Combinations Template

```java
public List<List<Integer>> combine(int n, int k) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(1, n, k, new ArrayList<>(), result);
    return result;
}

private void backtrack(int start, int n, int k, List<Integer> path,
                       List<List<Integer>> result) {
    if (path.size() == k) {
        result.add(new ArrayList<>(path));
        return;
    }

    for (int i = start; i <= n; i++) {
        path.add(i);
        backtrack(i + 1, n, k, path, result);
        path.remove(path.size() - 1);  // Backtrack
    }
}
```

Time & Space Complexity

Problem	Time Complexity	Space Complexity
Permutations	O(n! × n)	O(n)
Combinations	O(C(n,k) × k)	O(k)
Subsets	O(2^n × n)	O(n)
N-Queens	O(n!)	O(n)

Common Problem Patterns

1. Subsets

Problem: Generate all subsets of a set.

```java
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, int start, List<Integer> path, List<List<Integer>
    result.add(new ArrayList<>(path));
```

```
        for (int i = start; i < nums.length; i++) {
            path.add(nums[i]);
            backtrack(nums, i + 1, path, result);
            path.remove(path.size() - 1);
        }
    }
```

2. Combination Sum

Problem: Find combinations that sum to target (can reuse elements).

```
public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(candidates, target, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] candidates, int remain, int start,
                       List<Integer> path, List<List<Integer>> result) {
    if (remain == 0) { result.add(new ArrayList<>(path)); return; }
    if (remain < 0) return;

    for (int i = start; i < candidates.length; i++) {
        path.add(candidates[i]);
        backtrack(candidates, remain - candidates[i], i, path, result);
        path.remove(path.size() - 1);
    }
}
```

3. N-Queens

Problem: Place n queens on n×n board so no two attack each other.

```
public List<List<String>> solveNQueens(int n) {
    List<List<String>> result = new ArrayList<>();
    backtrack(n, 0, new int[n], result);
    return result;
}

private void backtrack(int n, int row, int[] queens, List<List<String>> result) {
    if (row == n) {
        result.add(buildBoard(queens, n));
        return;
    }
    for (int col = 0; col < n; col++) {
        if (isValid(queens, row, col)) {
            queens[row] = col;
            backtrack(n, row + 1, queens, result);
```

```
            }
        }
    }

    private boolean isValid(int[] queens, int row, int col) {
        for (int i = 0; i < row; i++) {
            if (queens[i] == col || Math.abs(queens[i] - col) == row - i) return false;
        }
        return true;
    }
```

4. Word Search

Problem: Find if word exists in 2D grid.

```
    public boolean exist(char[][] board, String word) {
        for (int i = 0; i < board.length; i++) {
            for (int j = 0; j < board[0].length; j++) {
                if (dfs(board, word, i, j, 0)) return true;
            }
        }
        return false;
    }

    private boolean dfs(char[][] board, String word, int i, int j, int k) {
        if (k == word.length()) return true;
        if (i < 0 || i >= board.length || j < 0 || j >= board[0].length) return false;
        if (board[i][j] != word.charAt(k)) return false;

        char temp = board[i][j];
        board[i][j] = '#';
        boolean found = dfs(board, word, i+1, j, k+1) || dfs(board, word, i-1, j, k+1) |
                        dfs(board, word, i, j+1, k+1) || dfs(board, word, i, j-1, k+1);
        board[i][j] = temp;
        return found;
    }
```

← Previous: Tree Algorithms | Back to Main Guide | Next: Trie & String Algorithms →